

Monte Carlo Tree Diffusion for System 2 Planning

Jaesik Yoon^{1,2} Hyeonseo Cho¹ Doojin Baek¹ Yoshua Bengio³ Sungjin Ahn^{1,4}

Abstract

Diffusion models have recently emerged as a powerful tool for planning. However, unlike Monte Carlo Tree Search (MCTS)—whose performance naturally improves with additional test-time computation (TTC)—standard diffusion-based planners offer only limited avenues for TTC scalability. In this paper, we introduce Monte Carlo Tree Diffusion (MCTD), a novel framework that integrates the generative strength of diffusion models with the adaptive search capabilities of MCTS. Our method reconceptualizes denoising as a tree-structured process, allowing partially denoised plans to be iteratively evaluated, pruned, and refined. By selectively expanding promising trajectories while retaining the flexibility to revisit and improve suboptimal branches, MCTD achieves the benefits of MCTS such as controlling exploration-exploitation trade-offs within the diffusion framework. Empirical results on challenging long-horizon tasks show that MCTD outperforms diffusion baselines, yielding higher-quality solutions as TTC increases.

1. Introduction

Diffusion models have recently emerged as a powerful approach to planning, enabling the generation of complex trajectories by modeling trajectory distributions using large-scale offline data (Janner et al., 2022; Ajay et al., 2022; Zhou et al., 2024; Chen et al., 2024a;c;b). Unlike traditional autoregressive planning methods, diffusion-based planners, such as Diffuser (Janner et al., 2022), generate entire trajectories holistically through a series of denoising steps, eliminating the need for a forward dynamics model. This approach effectively addresses key limitations of forward models, such as poor long-term dependency modeling and error accumulation (Hafner et al., 2022; Hamed et al., 2024), making it particularly well-suited for planning tasks with

long horizons or sparse rewards.

Despite their strengths, it remains uncertain how diffusion-based planners can effectively enhance planning accuracy by utilizing additional test-time compute (TTC)—a property referred to as TTC scalability. One potential approach is to increase the number of denoising steps or, alternatively, draw additional samples (Zhou et al., 2024). However, it is known that performance gains from increasing denoising steps plateau quickly (Karras et al., 2022; Song et al., 2020a;b), and independent random searches with multiple samples are highly inefficient as they fail to leverage information from other samples. Moreover, how to effectively manage the exploration-exploitation tradeoff within this framework also remains unclear.

In contrast, Monte Carlo Tree Search (MCTS) (Coulom, 2006), a widely adopted planning method, demonstrates robust TTC scalability. By leveraging iterative simulations, MCTS refines decisions and adapts based on exploratory feedback, making it highly effective in improving planning accuracy as more computation is allocated. This capability has established MCTS as a cornerstone in many System 2 reasoning tasks, such as mathematical problem-solving (Guan et al., 2025; Zhang et al., 2024a) and program synthesis (Brandfonbrener et al., 2024). However, unlike diffusion-based planners, traditional MCTS relies on a forward model for tree rollouts, inheriting its limitations including losing global consistency. In addition to being restricted to discrete action spaces, the resulting search tree can grow excessively large in both depth and width. This leads to significant computational demands, particularly in scenarios involving long horizons and large action spaces.

This raises a crucial question: *how can we combine the strengths of Diffuser and MCTS to overcome their limitations and enhance the TTC scalability of diffusion-based planning?* To address this, we propose Monte Carlo Tree Diffusion (MCTD), a framework that integrates diffusion-based trajectory generation with the iterative search capabilities of MCTS for more efficient and scalable planning.

MCTD builds on three key innovations. First, it restructures denoising into a tree-based rollout process, enabling semi-autoregressive causal planning while maintaining trajectory coherence. Second, it introduces guidance levels as meta-actions to dynamically balance exploration and exploitation,

¹KAIST ²SAP ³Mila ⁴New York University. Correspondence to: Jaesik Yoon <jaesik.yoon@kaist.ac.kr>, Sungjin Ahn <sungjin.ahn@kaist.ac.kr>.

ensuring adaptive and scalable trajectory refinement within the diffusion framework. Third, it employs fast jumpy denoising as a simulation mechanism, efficiently estimating trajectory quality without costly forward model rollouts. These innovations enable the four steps of MCTS (Selection, Expansion, Simulation, and Backpropagation) within diffusion planning, effectively bridging structured search with generative modeling. Experimental results show that MCTD outperforms existing approaches in long-horizon tasks, achieving superior scalability and solution quality.

The main contributions of this paper are as follows: First, to the best of our knowledge, this is the first work to propose an MCTS-integrated diffusion planning framework that explicitly incorporates the four steps of MCTS, providing an effective TTC scaling method for diffusion models. Second, we introduce three key innovations: Denoising as Tree-Rollout, Guidance Levels as Meta-Actions, and Jumpy Denoising as Fast Simulation. Lastly, we present experimental results demonstrating the effectiveness of MCTD.

2. Preliminaries

2.1. Diffuser: Diffusion Models for Planning

Diffuser (Janner et al., 2022) addresses long-horizon decision-making by treating entire trajectories as a matrix

$$\mathbf{x} = \begin{bmatrix} s_0 & s_1 & \dots & s_T \\ a_0 & a_1 & \dots & a_T \end{bmatrix},$$

where s_t and a_t denote the state and action at time t , respectively. A *diffusion process* is then trained to iteratively remove noise from samples of $\mathbf{x}$, ultimately producing coherent trajectories. In practice, this corresponds to reversing a forward noise-injection procedure by learning a denoiser $p_\theta(\mathbf{x})$ over the trajectory space. Since p_θ by itself does not encode reward or other task objectives, Diffuser optionally incorporates a heuristic or learned guidance function $J_\phi(\mathbf{x})$. This function predicts the return or value of a partially denoised trajectory, thereby biasing the sampling distribution:

$$\tilde{p}_\theta(\mathbf{x}) \propto p_\theta(\mathbf{x}) \exp(J_\phi(\mathbf{x})). \quad (1)$$

Accordingly, at each denoising step, gradient information from J_ϕ nudges the model toward trajectories that appear both feasible (as learned from the offline data) and promising with respect to returns, in a manner akin to classifier guidance in image diffusion (Dhariwal & Nichol, 2021).

Diffusion Forcing (Chen et al., 2024a) extends the above framework by allowing tokenization of $\mathbf{x}$. This tokenization allows each token to be denoised at a different noise level, enabling partial denoising of segments where uncertainty is high (e.g., future plans), without requiring a complete transition from full noise to no noise across the entire trajectory. Such token-level control is particularly beneficial

in domains that demand causal consistency, such as long-horizon planning problems.

2.2. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a planning algorithm that combines tree search with stochastic simulations to effectively balance exploration and exploitation (Coulom, 2006). Typically, MCTS proceeds in four stages: *selection*, *expansion*, *simulation*, and *backpropagation*. During *selection*, the algorithm starts at a root node corresponding to the current state (or partial plan) and descends the tree according to a policy such as Upper Confidence Bounds for Trees (UCT) (Kocsis & Szepesvári, 2006), aiming to balance choose the most promising child at each step or explore unexperienced states. Once a leaf or expandable node is reached, *expansion* adds new child nodes representing previously unexplored actions or plans.

Following expansion, *simulation* estimates the value of a newly added node by sampling a sequence of actions until reaching a terminal condition or a predefined rollout depth. The resulting outcome (e.g., cumulative reward in a Markov Decision Process) then updates the node’s estimated value through *backpropagation*, propagating this information to all ancestor nodes in the tree. Over multiple iterations, MCTS prunes unpromising branches and refines its value estimates for promising subtrees, guiding the search toward more effective solutions.

3. Monte Carlo Tree Diffusion

In this section, we first outline the key concepts that enable MCTD planning: (1) Denoising as Tree-Rollout, (2) Guidance Levels as Meta-Actions, and (3) Jumpy Denoising as Simulation. These innovations form the foundation of MCTD, bridging the gap between traditional tree search methods and diffusion-based planning. We then describe the MCTD process in terms of the four steps: Selection, Expansion, Simulation, and Backpropagation.

3.1. Denoising as Tree-Rollout

Traditional MCTS operates on individual states, leading to deep search trees and significant scalability challenges. Because each node in the tree represents a single state, the depth of the tree increases linearly with the planning horizon, resulting in an exponential growth of the search space. Furthermore, it lacks the holistic perspective of the entire trajectory that diffusion-based planners inherently provide. On the other hand, Diffuser does not offer the tree-like structure necessary for intermediate decision-point searches that effectively balance exploitation and exploration.

To address this issue, we first introduce the *Denoising as Tree-Rollout* process by leveraging the semi-autoregressive

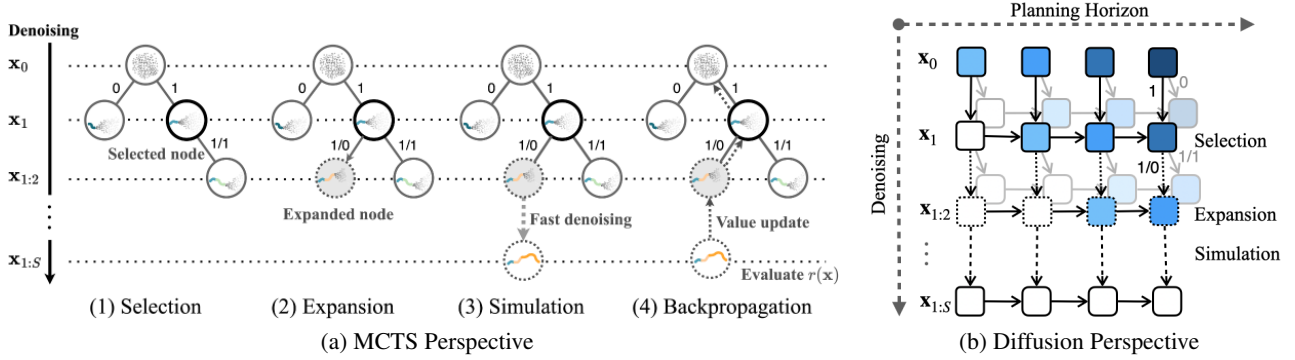


Figure 1. Two perspectives on Monte Carlo Tree Diffusion (MCTD). **(a) MCTS Perspective:** The four steps of an MCTD round—*Selection*, *Expansion*, *Simulation*, and *Backpropagation*—are illustrated on a partial denoising tree. Each node corresponds to a partially denoised sub-trajectory, and edges are labeled with binary guidance levels (0 = no guidance, 1 = guided). After a new node is expanded, “jumpy” denoising is performed to quickly estimate its value, which is then backpropagated along the path in the tree. **(b) Diffusion Perspective:** The same process is viewed as partial denoising across both denoising depth (vertical axis) and planning horizon (horizontal axis). Each colored block represents a partially denoised plan at a specific noise level, with darker shades indicating higher noise. Different expansions (0 or 1) create branches in the forward planning direction, representing alternative trajectory refinements. Notably, the entire row is denoised simultaneously, but with varying denoising levels. The MCTD framework unifies these two perspectives.

denoising process (Chen et al., 2024a). Specifically, we partition the full trajectory $\mathbf{x} = (x_1, x_2, \dots, x_N)$ into S subplans, $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_S)$ such that $\cap_s \mathbf{x}_s = \emptyset$. Unlike the standard diffuser, where all subplans share the same global denoising schedule, our approach allows assigning independent denoising schedules to each subplan. By applying faster denoising to earlier subplans and slower denoising to later subplans, the process effectively becomes causal and semi-autoregressive. This allows the denoising process to determine the future conditioned on the already determined past. Consequently, while *preserving the globally consistent and holistic generation advantages of Diffuser*, the denoising process approximates:

$$p(\mathbf{x}) \approx \prod_{s=1}^S p(\mathbf{x}_s | \mathbf{x}_{1:s-1}). \quad (2)$$

Notably, although this formulation appears autoregressive, it is still executed as a single denoising process by controlling noise levels across subplans (Chen et al., 2024a).

In MCTD, each subplan $\mathbf{x}_s$, which represents a *temporally extended state*, is treated as a node in the search tree, rather than using individual states x_n as nodes. This allows the tree to operate at a higher level of abstraction, improving efficiency and scalability. Denoising the entire plan $\mathbf{x}$ can be viewed as rolling out a sequence of these nodes through a single denoising process of Diffuser. Because $S \ll N$, the tree depth becomes much smaller than that in MCTS. For instance, in one of our planning experiments, we use $S = 5$ and $N = 500$.

3.2. Guidance Levels as Meta-Actions

In MCTS, constructing and searching the tree becomes computationally expensive in large action spaces and is fundamentally impractical for continuous action spaces. To resolve this issue, in MCTD we introduce a novel approach by redefining the exploration-exploitation trade-off in terms of *meta-actions*. While meta-actions can be any discrete decision that can control the exploration-exploitation, in this work we propose to implement it as *guidance levels* applied during the denoising process.

For simplicity, consider two guidance levels: GUIDE and NO_GUIDE. In MCTD, we observe that sampling from the prior distribution $p(\mathbf{x})$, i.e., using the standard diffusion sampler represents exploratory behavior as it does not attempt to achieve any goal but only imitates the prior behavior contained in the offline data. Thus, we correspond it to the NO_GUIDE meta-action. Conversely, sampling from a goal-directed distribution $p_g(\mathbf{x})$, such as through classifier-guided diffusion (Dhariwal & Nichol, 2021), represents exploitative behavior, and thus meta-action, GUIDE, is assigned. This steers the sampling process toward achieving a specific goal defined by a reward function $r_g(\mathbf{x})$.

Next, we integrate the concept of meta-actions with the tree-rollout denoising process described in Eqn. (2). To achieve this, we introduce a guidance schedule $\mathbf{g} = (g_1, \dots, g_S)$, which assigns a guidance meta-action $g_s \in \{\text{GUIDE}, \text{NO_GUIDE}\}$ to each corresponding subplan $\mathbf{x}_s$. If no guidance is chosen, the subplan is sampled from the explorative tree-rollout prior $p(\mathbf{x}_s | \mathbf{x}_{1:s-1})$ while a subplan with guidance is sampled from $p_g(\mathbf{x}_s | \mathbf{x}_{1:s-1})$ if the GUIDE meta-action is chosen. This allows the guidance levels to be

independently assigned to each subplan unlike the standard Diffuser assigning the guidance throughout both the whole trajectory and the whole denoising process.

By dynamically adjusting the guidance schedule $\mathbf{g}$, this enables the exploration-exploitation balancing at the level of subplans *within a single denoising process*. The extended tree-rollout denoising process effectively approximates:

$$p(\mathbf{x}|\mathbf{g}) \approx \prod_{s=1}^S p(\mathbf{x}_s | \mathbf{x}_{1:s-1}, g_s). \quad (3)$$

As a result, this approach enables efficient and scalable planning, even in complex or continuous action spaces.

3.3. Jumpy Denoising as Fast Simulation

In MCTS, evaluating a node far from a leaf node, where evaluation of the plan is feasible, is a critical requirement. This is typically addressed in one of two ways: using a fast forward dynamics model to roll out trajectories to the leaf node, which is computationally expensive, or approximating the node’s value via bootstrapping, offering faster but less accurate results. However, how to effectively incorporate these simulation strategies into the diffuser framework remains an open question.

In MCTD, we implement this simulation functionality using the *fast jumpy denoising* process based on DDIM (Song et al., 2020a). Specifically, when the tree-rollout denoising process has progressed up to the s -th subplan, the remaining steps are denoised quickly by skipping every C step:

$$\tilde{\mathbf{x}}_{s+1:S} \sim p(\mathbf{x}_{s+1:S} | \mathbf{x}_{1:s}, \mathbf{g}). \quad (4)$$

This produces a full trajectory $\tilde{\mathbf{x}} = (\mathbf{x}_{1:s}, \tilde{\mathbf{x}}_{s+1:S})$, which is then evaluated using the reward function $r(\tilde{\mathbf{x}})$. While this fast denoising process may introduce larger approximation errors, it is highly computationally efficient, making it well-suited for the simulation step in MCTD.

3.4. The Four Steps of an MCTD Round

Building on the description above, we detail how the four traditional steps of MCTS—Selection, Expansion, Simulation, and Backpropagation—are adapted and implemented in MCTD. This process is illustrated in Figure 1.

Selection. In MCTD, the selection process involves traversing the tree from the root node to a leaf or partially expanded node. At each step, a child node is chosen based on a selection criterion such as Upper Confidence Bound (UCB). Importantly, this step does not require computationally expensive denoising; it simply traverses the existing tree structure. Unlike traditional MCTS, MCTD nodes correspond to temporally extended states, enabling higher-level reasoning and reducing tree depth, which improves scalability. The guidance schedule $\mathbf{g}$ is dynamically adjusted during

Algorithm 1 Monte Carlo Tree Diffusion

```

1: procedure MCTD( $root, iterations$ )
2:   for  $i = 1$  to  $iterations$  do
3:      $node \leftarrow root$ 
4:     while ISFULLYEXPANDED( $node$ ) and not
5:       ISLEAF( $node$ ) do
6:        $node \leftarrow BESTUCTCHILD(node)$ 
7:     end while
8:     if ISEXANDABLE( $node$ ) then
9:        $g_s \leftarrow SELECTMETACTION(node)$ 
10:       $newSubplan \leftarrow DENOISESUBPLAN(node, g_s)$ 
11:       $child \leftarrow CREATENODE(newSubplan)$ 
12:      ADDCHILD( $node, child$ )
13:       $node \leftarrow child$ 
14:     end if
15:      $partial \leftarrow GETPARTIALTRAJECTORY(node)$ 
16:      $remaining \leftarrow FASTDENOISING(partial)$ 
17:      $fullPlan \leftarrow (partial, remaining)$ 
18:      $reward \leftarrow EVALUATEPLAN(fullPlan)$ 
19:     while  $node \neq null$  do
20:        $node.visitCount \leftarrow node.visitCount + 1$ 
21:        $node.value \leftarrow node.value + reward$ 
22:       UPDATEMETACTIONSCHEDULE(
23:          $node, reward$ )
24:        $node \leftarrow node.parent$ 
25:     end while
26:   end for
27:   return BESTCHILD( $root$ )
28: end procedure

```

this step by UCB to balance exploration (NO_GUIDE) and exploitation (GUIDE).

Expansion. Once a leaf or partially expanded node is selected, the expansion step generates new child nodes by extending the current partially denoised trajectory. Each child node corresponds to a new subplan generated using the diffusion model. Depending on the meta-action g_s , the subplan is either sampled from the exploratory prior distribution $p(\mathbf{x}_s | \mathbf{x}_{1:s-1})$ or the goal-seeking distribution $p_g(\mathbf{x}_s | \mathbf{x}_{1:s-1})$. The meta-action controls which behavior is used for expansion, and the newly generated node is added to the tree as an extension of the current trajectory. Importantly, the guidance levels are not restricted to binary choices. For instance, it is possible to generalize the meta-actions to multiple guidance levels, such as {ZERO, LOW, MEDIUM, HIGH}, offering finer control over the balance between exploration and exploitation during the expansion process.

Simulation. Simulation in MCTD is implemented via the fast jumpy denoising. When a node is expanded, the remainder of the trajectory is quickly completed by using the fast denoising as described in Section 3.3. The resulting plan $\tilde{\mathbf{x}}$ is then given to the plan evaluator $r(\tilde{\mathbf{x}})$. This approach maintains computational efficiency while providing a sufficient estimate of the plan’s quality.

Backpropagation. After the simulation step, the reward

obtained from evaluating the complete plan is backpropagated through the tree to update the value estimates of all parent nodes along the path to the root. In MCTD, this backpropagation process also updates the meta-action-based guidance schedules, enabling the tree to dynamically adjust the exploration-exploitation balance for future iterations. This ensures that promising trajectories, as indicated by their rewards, are prioritized, while sufficient exploration is maintained to prevent premature convergence.

4. Related Works

Diffusion models (Sohl-Dickstein et al., 2015) have recently shown significant promise for long-horizon trajectory planning, particularly in settings with sparse rewards, by learning to generate entire sequences rather than stepwise actions (Janner et al., 2022; Chen et al., 2024a; Ajay et al., 2022; Zhou et al., 2024; Chen et al., 2024c;b; Chen et al.). Various enhancements have been proposed to extend their capabilities. For instance, Chen et al. (2024a) incorporate causal noise scheduling for semi-autoregressive planning, and other works introduce hierarchical structures (Chen et al., 2024c;b; Chen et al.), low-level value learning policies (Chen et al., 2024b), or classifier-free guidance (Ajay et al., 2022; Zhou et al., 2024). Despite these developments, the explicit interplay between exploration and exploitation in diffusion sampling has received relatively little attention.

Chen et al. (2024a) further propose Monte Carlo Guidance (MCG) to leverage multiple sub-plans and average their guidance signals, thereby biasing denoising toward higher-reward outcomes. While this can encourage planning toward an expected reward over multiple rollouts, it does not implement an explicit search mechanism. The detailed discussion is in Appendix C. Similarly, Anonymous (2024) apply a discrete diffusion denoising process to tasks such as chess, implicitly modeling MCTS without explicit tree expansion.

MCTS (Coulom, 2006) has achieved impressive results across various decision-making problems, particularly when combined with learned policies or value networks (Silver et al., 2016; Schrittwieser et al., 2020). It has also been applied to System 2 reasoning in Large Language Models (LLMs) to enhance structured problem-solving (Xiang et al., 2025; Yao et al., 2024; Zhang et al., 2024b). However, to the best of our knowledge, this work is the first to integrate tree search with diffusion models for full trajectory generation, bridging structured search with generative planning.

5. Experiments

We evaluate the proposed approach, MCTD, on a suite of tasks from the Offline Goal-conditioned RL Benchmark (OGBench) (Park et al., 2024), which spans diverse domains such as maze navigation with multiple robot mor-

phologies (e.g., point-mass or ant) and robot-arm manipulation. Our chosen tasks—point and antmaze navigation, multi-cube manipulation, and a newly introduced visual pointmaze—jointly assess a planner’s ability to handle *long-horizon planning*, *sequential manipulation*, and *partial visual observability*. In the visual pointmaze, an agent perceives RGB image observations of the 3D environment, thereby testing each method’s resilience to partial observability and the ability to handle image-based planning.

5.1. Baselines

We compare MCTD against several baselines, each employing a distinct strategy for test-time computation. First, we include the standard single-shot **Diffuser** (Janner et al., 2022), which plans only at the start of an episode and executes that plan without further adjustments. Next, we consider two methods that allocate additional test-time compute differently. **Diffuser-Replanning** periodically replans at fixed intervals, allowing partial corrections during the episode. This enables us to gauge the benefits of iterative replanning without a full tree search. **Diffuser-Random Search** (Zhou et al., 2024) generates multiple random trajectories in parallel and selects the highest-scoring one according to the same reward used by MCTD. While this increases test-time sampling and applies multiple guidance levels for different samples, it lacks the systematic expansion and pruning of a tree-search algorithm. We also compare our method to **Diffusion Forcing** (Chen et al., 2024a), which introduces a causal denoising schedule that enables semi-autoregressive trajectory generation. This comparison helps isolate the benefits of semi-autoregressive denoising from our full tree-structured approach. Further implementation details about all baselines can be found in Appendix A.1.

5.2. Maze Navigation with Point-Mass and Ant Robots

We begin with the pointmaze and antmaze tasks from OGBench, featuring mazes of varying sizes (medium, large, and giant). The agent is rewarded for reaching a designated goal region, and the relevant dataset (*navigate*) comprises long-horizon trajectories that challenge each method’s capacity for exploration. As in prior work (Janner et al., 2022), we employ a heuristic controller for the pointmaze environment, whereas the antmaze environment uses a value-learning policy (Chen et al., 2024b) for low-level control. Appendix A.5 provides more details on this task evaluation.

Results. Table 1 presents success rates across medium, large, and giant mazes for both point-mass and ant robots. **We can see that MCTD consistently surpasses other methods by a large margin.** Notably, Diffuser-Random Search, despite using roughly the same number of denoising steps as MCTD, demonstrates no improvement over one-shot Diffuser, underscoring the importance of tree-

Table 1. Long-Horizon Maze Results. Success rates (%) on pointmaze and antmaze environments with medium, large, and giant mazes for the *navigate* datasets. Each cell shows the mean $\pm$ standard deviation.

Environment	Dataset	Diffuser			Diffusion Forcing	MCTD
		Base	Replanning	Random Search		
pointmaze	medium-navigate-v0	58 $\pm$ 6	60 $\pm$ 0	60 $\pm$ 9	65 $\pm$ 16	100 $\pm$ 0
	large-navigate-v0	44 $\pm$ 8	40 $\pm$ 0	34 $\pm$ 13	74 $\pm$ 9	98 $\pm$ 6
	giant-navigate-v0	0 $\pm$ 0	0 $\pm$ 0	4 $\pm$ 8	50 $\pm$ 10	100 $\pm$ 0
antmaze	medium-navigate-v0	36 $\pm$ 15	40 $\pm$ 18	48 $\pm$ 10	90 $\pm$ 10	100 $\pm$ 0
	large-navigate-v0	14 $\pm$ 16	26 $\pm$ 13	20 $\pm$ 0	57 $\pm$ 6	98 $\pm$ 6
	giant-navigate-v0	0 $\pm$ 0	0 $\pm$ 0	4 $\pm$ 8	24 $\pm$ 12	94 $\pm$ 9

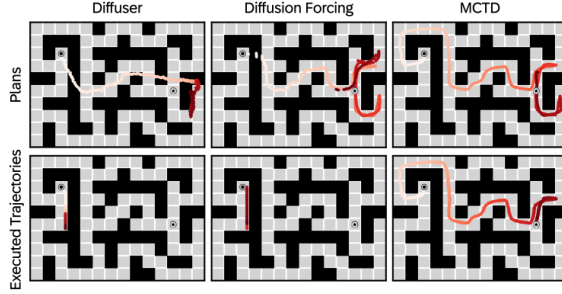


Figure 2. Comparison of generated plans and actual rollouts for three planners—Diffuser, Diffusion Forcing, and MCTD. While Diffuser and Diffusion Forcing fail to produce successful trajectory plans, MCTD succeeds by refining its plan adaptively.

based systematic branching over random sampling. Diffuser-Replanning similarly fails to outperform the base Diffuser, whereas Diffusion Forcing exhibits higher success rates in larger mazes, indicating benefits from a semi-autoregressive schedule for long horizons. However, only MCTD achieves near-perfect performance on the most challenging (large, giant) mazes, thanks to its ability to refine partial trajectories and avoid dead-ends or inconsistent plan segments. This process is visualized in Figure 3. The performance on the giant map is impressive, because in the map, there are lots of detour paths to reach the goal as shown in Figure 2. Comparing with Diffuser-Random Search further highlights MCTD’s efficiency, since it offers substantially better outcomes under the same computational budget.

In the antmaze domain, **MCTD continues to perform nearly perfectly, while baseline performance generally degrades more significantly than in pointmaze**, except for the medium-sized settings under Diffusion Forcing. A principal reason for the baseline’s degradation is that the value learning policy (Wang et al., 2022) struggles in high-dimensional control tasks. Even the performance degradations exist due to inaccurate execution, MCTD consistently outperforms baselines and the gap is larger as the maze size is larger.

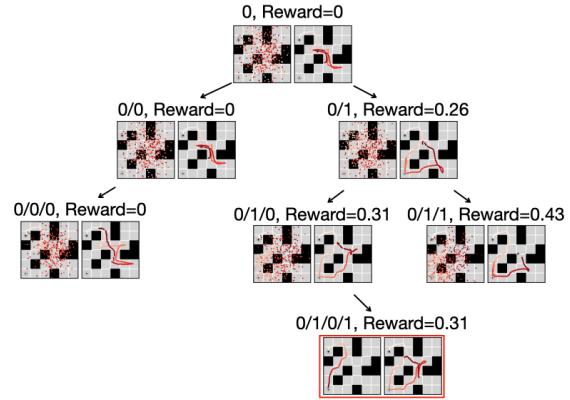


Figure 3. Visualization of the MCTD tree-search process with a binary guidance set {NO_GUIDE, GUIDE} on a pointmaze-medium task. Each node corresponds to a partially denoised trajectory, where the left image shows the noisy partial plan and the right image shows the plan after fast denoising. The search expands child nodes by selecting either NO_GUIDE or GUIDE, evaluates each newly generated plan, and ultimately converges on the highlighted leaf as the solution.

5.3. Robot Arm Cube Manipulation

For multi-cube manipulation tasks from OGBench, a robot arm must move one to four cubes to specific table locations as shown in Figure 4a. Increasing the number of cubes grows both the planning horizon and the combinatorial complexity. While the planner (MCTD or a baseline) issues high-level positions for the cubes, a value-learning policy (Wang et al., 2022) executes local actions. We augment MCTD with *object-wise guidance* by selecting which cube is manipulated at each step, avoiding simultaneous movements of multiple cubes. We note that the object-wise guidance is particularly suited to MCTD because the tree-expansion mechanism naturally supports discrete “meta-actions” at each node, such as selecting which object to move next. Single-pass diffusion methods, by contrast, attempt to produce a single, holistic trajectory in one go, leaving no opportunity to decide object-by-object. This guidance and additional task-specific reward shaping are described in Appendix A.6.

Results. Table 2 shows success rates for one to four cubes. All methods perform comparably on single-cube tasks, but

Table 2. **Robot Arm Cube Manipulation Results.** Success rates (%) for single, double, triple, and quadruple cube tasks in OGBench. Parenthetical values in the MCTD-Replanning column denote performance when using a DQL performer trained only on the single-cube dataset.

Dataset	Diffuser	Diffuser-Replanning	Diffusion Forcing	MCTD	MCTD-Replanning
single-play-v0	78 $\pm$ 23	92 $\pm$ 13	100 $\pm$ 0	98 $\pm$ 6	100 $\pm$ 0
double-play-v0	12 $\pm$ 10	12 $\pm$ 13	18 $\pm$ 11	22 $\pm$ 11	50 $\pm$ 16 (78 $\pm$ 11)
triple-play-v0	8 $\pm$ 10	4 $\pm$ 8	16 $\pm$ 8	0 $\pm$ 0	6 $\pm$ 9 (40 $\pm$ 21)
quadruple-play-v0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0 (24 $\pm$ 8)

Table 3. **Visual Pointmaze Results.** Success rates (%) on partially observable, image-based mazes of medium and large sizes.

Dataset	Diffuser	Diffuser-Replanning	Diffusion Forcing	MCTD	MCTD-Replanning
medium-navigate-v0	8 $\pm$ 13	8 $\pm$ 10	66 $\pm$ 32	82 $\pm$ 18	90 $\pm$ 9
large-navigate-v0	0 $\pm$ 0	0 $\pm$ 0	8 $\pm$ 12	0 $\pm$ 0	20 $\pm$ 21

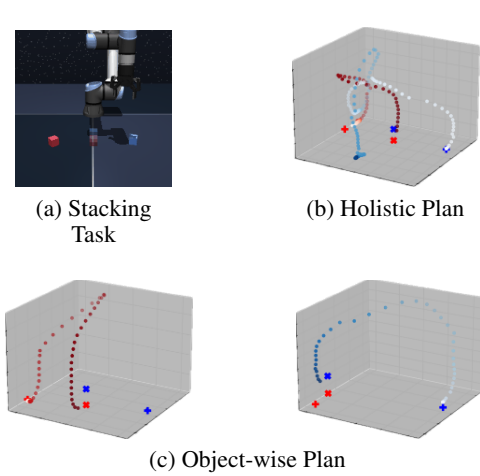


Figure 4. (a) Illustration of the robot arm cube manipulation task. (b) A holistic plan for moving two cubes, where entanglement between their partial trajectories leads to an inaccurate movement (blue cube). (c) An object-wise plan produced by MCTD-Replanning, which avoids entanglement and achieves more reliable control over each cube’s movement.

performance drops significantly for multi-cube scenarios, especially for standard diffusion baselines that struggle to decide cube ordering and handle subtasks such as stacking. However, **MCTD-Replanning shows significant performance gaps from other models for multi-cube scenarios.** MCTD exhibits moderate gains (e.g., 22% on two cubes), though it suffers from holistic plan entanglements when multiple objects are involved as shown in Figure 4b. To address this, *MCTD-Replanning* periodically replans, effectively separating each cube’s movements as shown in Figure 4c. This boosts success on the double-cube problem from 22% to 50%, and also benefits tasks with more cubes. Another challenge arises from the value-learning policy, which generalizes poorly as the number of cubes increases. Even if MCTD generates a suitable global plan, flawed local control undermines the final outcome. Interestingly, training the policy solely on single-cube data still allows MCTD-Replanning to solve a portion (24%) of quadruple-cube scenarios, illustrating the potential of object-wise guidance

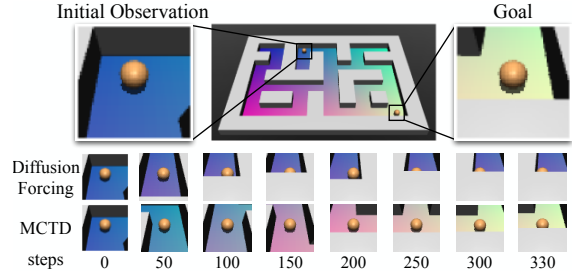


Figure 5. Task and interaction examples in the visual pointmaze. The agent operates under partial observability, receiving pixel-based initial and goal images. In the top row, Diffusion Forcing struggles to reach the goal due to a focus on exploitation, failing to sufficiently explore the environment. In contrast, MCTD (bottom row) effectively balances exploration and exploitation, successfully navigating to the goal.

and iterative replanning.

5.4. Visual Pointmaze

To evaluate our method in image-based planning, particularly under partial observability, we introduce a visual pointmaze environment where the agent receives top-down pixel images of the initial and goal states, as shown in Figure 5. Unlike the visual antmaze in OGBench, which involves complex locomotion, this environment allows us to focus on how well the planner operates with raw visual inputs without relying on ground-truth states.

We pre-encode observations with a Variational Autoencoder (VAE) (Kingma & Welling, 2013) to generate latent states suitable for diffusion-based planning. Because Diffuser’s default heuristic controller is inapplicable to purely visual inputs, we design an *inverse dynamics* model that maps consecutive latent observations to actions, $\hat{a}_t = \text{InvDyna}((x_{t-1}, x_t), x_{t+1})$. For guidance, we employ a position estimator trained on ground-truth coordinates; however, this estimator does not update the underlying VAE, ensuring that the planner remains in a partially observable regime. The details for this task setting are discussed in Appendix A.7.

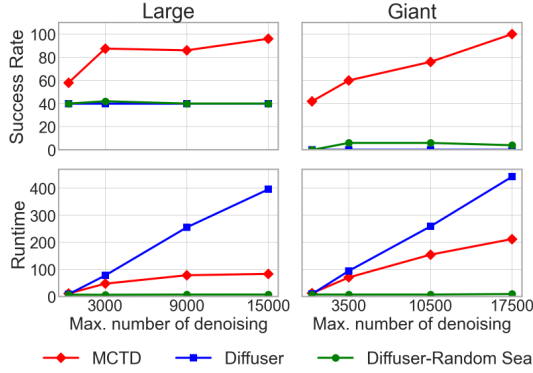


Figure 6. Success rates (%) and wall-clock runtime (Sec.) as the maximum number of denoising steps (test-time budget) increases for large and giant pointmaze tasks

Results. Table 3 compares success rates on medium and large versions of the visual pointmaze. **MCTD and MCTD-Replanning show better performance than baselines on image-based, partial observable tasks.** MCTD outperforms Diffuser baselines and Diffusion Forcing in the medium maze, presumably due to its tree-structured exploration, which better balances exploration and exploitation under perceptual partial observability. In the large maze, single-pass planners fail almost entirely, highlighting the difficulty of long-horizon visual planning. MCTD-Replanning improves performance by reevaluating partial plans mid-trajectory, though success remains modest overall. This result underscores the inherent challenge of scaling diffusion-based methods to extended horizons with partial observational visual data and suggests that future work may need more sophisticated visual encoders or hierarchical guidance strategies.

5.5. Test-Time Compute Scalability & Time Complexity

We examine how each planner leverages additional *test-time computation* by varying the maximum number of denoising steps and measuring both *success rate* and *runtime* on the large and giant pointmaze tasks (Figure 6). **As the TTC budget grows, MCTD consistently achieves high success rates—ultimately nearing perfection in giant mazes.** In contrast, simply increasing denoising steps (Diffuser) or drawing more random samples (Diffuser-Random Search) yields modest returns, suggesting limited test-time scalability.

MCTD incurs additional overhead from maintaining and expanding a tree—leading to a larger runtime as the budget grows. For instance, Diffuser-Random Search shows moderate runtime growth because multiple samples can be denoised in parallel batches, which is not directly feasible in MCTD’s tree-structured approach. Nonetheless, MCTD can terminate early whenever a successful plan is found, thereby limiting unnecessary expansions. As a result, on the large maze, MCTD’s runtime quickly saturates once

the task can be solved with a smaller search depth, rather than fully consuming the available budget. A similar pattern emerges in the giant maze: as success rates increase under higher budgets, the runtime growth for MCTD remains sub-linear, indicating that many expansions terminate early once suitable trajectories are located.

6. Limitations & Discussion

While MCTD enhances test-time compute scalability by integrating search-based planning with diffusion models, it remains computationally expensive due to its System 2-style deliberate reasoning. Determining when to engage in structured planning versus relying on fast System 1 (model-free) planning is an open challenge, as expensive tree search may not always be necessary. Adaptive compute allocation based on task complexity or uncertainty could improve efficiency.

Another limitation is inefficiency in large-scale search spaces, where evaluating multiple trajectory hypotheses remains computationally demanding despite using low-dimensional meta-actions. A promising direction is amortizing search learning, where the system meta-learns from test-time search to improve exploration efficiency over time. Instead of treating initial exploration as random, MCTD could incorporate test-time learning mechanisms to refine its search dynamically. Additionally, self-supervised reward shaping could improve trajectory evaluation in sparse-reward settings. Optimizing parallelized denoising, differentiable tree search, and model-based rollouts could further enhance efficiency.

7. Conclusion

We introduced Monte Carlo Tree Diffusion (MCTD), a framework designed to combine the best of both worlds: the structured search of Monte Carlo Tree Search and the generative flexibility of diffusion planning to enhance the test-time compute scalability of System 2 planning. MCTD leverages meta-actions for adaptive exploration-exploitation, tree-structured denoising for efficient diffusion-based expansion, and fast jumpy denoising for rapid simulation. Experimental results demonstrate that MCTD outperforms existing approaches in various planning tasks, achieving superior scalability and solution quality. Future work will explore adaptive compute allocation, learning-based meta-action selection, and reward shaping to further enhance performance, paving the way for more scalable and flexible System 2 planning.

Acknowledgements

This research was supported by GRDC (Global Research Development Center) Cooperative Hub Program (RS-2024-00436165) and Brain Pool Plus Program (No. 2021H1D3A2A03103645) through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT.

Impact Statement

This work introduces Monte Carlo Tree Diffusion (MCTD), a framework that integrates diffusion models with Monte Carlo Tree Search (MCTS) for scalable, long-horizon planning. As a general-purpose planning framework, MCTD itself does not pose direct risks; however, care must be taken when applying it to safety-critical domains, where decision-making impacts human well-being or critical infrastructure. Depending on its implementation, MCTD could influence high-stakes decisions in areas such as autonomous systems, healthcare, or finance, necessitating robust oversight and alignment with ethical guidelines. Additionally, its computational demands raise considerations around energy efficiency and sustainability, highlighting the need for responsible AI deployment.

References

- Ajay, A., Du, Y., Gupta, A., Tenenbaum, J., Jaakkola, T., and Agrawal, P. Is conditional generative modeling all you need for decision-making? *arXiv preprint arXiv:2211.15657*, 2022.
- Anonymous. Implicit search via discrete diffusion: A study on chess. In *Submitted to The Thirteenth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=A9y3LFX4ds>. under review.
- Brandfonbrener, D., Henniger, S., Raja, S., Prasad, T., Loughridge, C. R., Cassano, F., Hu, S. R., Yang, J., Byrd, W. E., Zinkov, R., et al. Vermcts: Synthesizing multi-step programs using a verifier, a large language model, and tree search. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS'24*, 2024.
- Chen, B., Monso, D. M., Du, Y., Simchowicz, M., Tedrake, R., and Sitzmann, V. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *arXiv preprint arXiv:2407.01392*, 2024a.
- Chen, C., Hamed, H., Baek, D., Bengio, Y., and Ahn, S. Hierarchical multiscale diffuser for extendable long-horizon planning.
- Chen, C., Baek, J., Deng, F., Kawaguchi, K., Gulcehre, C., and Ahn, S. Plandq: Hierarchical plan orchestration via d-conductor and q-performer. *arXiv preprint arXiv:2406.06793*, 2024b.
- Chen, C., Deng, F., Kawaguchi, K., Gulcehre, C., and Ahn, S. Simple hierarchical planning with diffusion. *arXiv preprint arXiv:2401.02644*, 2024c.
- Coulom, R. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pp. 72–83. Springer, 2006.
- Dhariwal, P. and Nichol, A. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.
- Guan, X., Zhang, L. L., Liu, Y., Shang, N., Sun, Y., Zhu, Y., Yang, F., and Yang, M. rstar-math: Small llms can master math reasoning with self-evolved deep thinking. *arXiv preprint arXiv:2501.04519*, 2025.
- Hafner, D., Lee, K.-H., Fischer, I., and Abbeel, P. Deep hierarchical planning from pixels. *arXiv preprint arXiv:2206.04114*, 2022.
- Hamed, H., Kim, S., Kim, D., Yoon, J., and Ahn, S. Dr. strategy: Model-based generalist agents with strategic dreaming. *arXiv preprint arXiv:2402.18866*, 2024.
- Janner, M., Du, Y., Tenenbaum, J. B., and Levine, S. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.
- Karras, T., Aittala, M., Aila, T., and Laine, S. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35: 26565–26577, 2022.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Kocsis, L. and Szepesvári, C. Bandit based monte-carlo planning. In *European conference on machine learning*, pp. 282–293. Springer, 2006.
- Park, S., Frans, K., Eysenbach, B., and Levine, S. Ogbench: Benchmarking offline goal-conditioned rl. *arXiv preprint arXiv:2410.20092*, 2024.
- Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839): 604–609, 2020.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pp. 2256–2265. PMLR, 2015.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020a.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020b.
- Wang, Z., Hunt, J. J., and Zhou, M. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.
- Xiang, V., Snell, C., Gandhi, K., Albalak, A., Singh, A., Blagden, C., Phung, D., Rafailov, R., Lile, N., Mahan, D., et al. Towards system 2 reasoning in llms: Learning how to think with meta chain-of-thought. *arXiv preprint arXiv:2501.04682*, 2025.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- Zhang, D., Huang, X., Zhou, D., Li, Y., and Ouyang, W. Accessing gpt-4 level mathematical olympiad solutions via monte carlo tree self-refine with llama-3 8b. *arXiv preprint arXiv:2406.07394*, 2024a.
- Zhang, D., Zhoubian, S., Hu, Z., Yue, Y., Dong, Y., and Tang, J. Rest-mcts*: Llm self-training via process reward guided tree search. *arXiv preprint arXiv:2406.03816*, 2024b.
- Zhou, G., Swaminathan, S., Raju, R. V., Guntupalli, J. S., Lehrach, W., Ortiz, J., Dedieu, A., Lázaro-Gredilla, M., and Murphy, K. Diffusion model predictive control. *arXiv preprint arXiv:2410.05364*, 2024.

A. Experiment Details

A.1. Baselines

We compare MCTD against several baselines, each highlighting different ways to leverage test-time computing on diffusion models or broader planning strategies:

- **Diffuser** (Janner et al., 2022): We use the standard single-shot Diffuser as our primary diffusion baseline. It generates a full trajectory by denoising from maximum to zero noise in one pass, guided by a return-predicting function J_ϕ . While it can produce coherent plans, it lacks any mechanism to adapt or refine these plans at test time.
- **Diffuser-Replanning**: To partially address the lack of iterative search, we evaluated a replanning variant of Diffuser. After an initial denoised plan is sampled, the policy is periodically re-invoked at predefined intervals (e.g., every 10 time steps), using the agent’s new state as the “start” for a fresh diffusion process. This strategy leverages additional test-time compute by attempting incremental corrections, but does not share information across different replans.
- **Diffuser-Random Search**: We consider a random-search variant of Diffuser that generates multiple trajectories from the randomly noised plans and picks the best candidate according to a heuristic or learned value function. This approach can be seen as a “Sample-Score-Rank” method proposed in (Zhou et al., 2024), increasing test-time compute by drawing more samples, but lacking the systematic exploration or adaptation found in tree-search paradigms.
- **Diffusion Forcing**: Diffusion Forcing (Chen et al., 2024a) extends Diffuser by introducing a tokenized, causal noise schedule. Subplans of the trajectory can be denoised at different rates, allowing partial re-sampling of specific time segments without re-optimizing the entire plan. In our experiments, we evaluate Diffusion Forcing with the Transformer backbone provided in their official repository, which performed better in our preliminary tests. This method helps isolate the benefit of tokenized partial denoising from our tree-structured approach. We note that Diffusion Forcing baselines are basically designed to do replanning, so we follow the setting in this evaluation.

A.2. Heuristic Controller, Value-Learning Policy and Inverse Dynamics Model

A long-standing challenge in diffusion-based planning is the difficulty of preserving global trajectory coherence while managing local, high-dimensional state-action control (Chen et al., 2024a;b). For example, PlanDQ (Chen et al., 2024b) couples a high-level diffusion planner with a learned low-level policy. Similarly, our approach focuses the diffusion planner on lower-dimensional, representative state information (e.g., object or agent positions), while delegating detailed action inference to the heuristic controller or value-learning policy or inverse dynamics model. The details of this integration is discussed in Algorithm 10. Such a hierarchical design allows MCTD and baseline planners to concentrate on broader strategic decisions while leaving the finer-grained execution details to the heuristic controller or value-learning policy or inverse dynamics model.

A.3. Model Hyperparameters

For reproducibility, we provide details on the hyperparameters used in our experiments. These settings were chosen based on prior work and empirical tuning to ensure stable training and evaluation across all tasks. The almost same hyperparameters were applied consistently across tasks, except in cases where task-specific configurations are required. They will be discussed in each task section.

A.3.1. DIFFUSER

We build our tested Diffuser on the official implementation, which is available <https://github.com/jannerm/diffuser>. While keeping the core architecture and training procedure intact, we made the following modifications to align with our experimental setup:

- **Guidance Function**: For pointmaze tasks, Diffuser originally applied the goal-inpainting guidance, but in our evaluation, we employed a distance-based reward function to ensure fair comparison across baselines.
- **Replanning Variant**: To evaluate the impact of test-time iterative refinement, we implemented a replanning strategy that periodically re-samples trajectories based on the agent’s updated state.

Table 4. Diffuser Hyperparameters

Hyperparameter	Value
Learning Rate	$2e - 4$
EMA Decay	0.995
Precision in Training/Inference	32
Batch Size	32
Max Training Steps	20000
Planning Horizon	Different for each task, please check in the task sections
Open Loop Horizon	50 for Diffuser-Replanning, otherwise planning horizon
Guidance Scale	0.1
Beta Schedule	Cosine
Diffusion Model Objective	x_0 -prediction
U-Net Depth	4
Kernel Size	5
The Number of Channels	32, 128, 256

- **Random Search Variant:** To evaluate the impact of test-time compute scaling with more samples, we implemented Diffuser version "Sample-Score-Rank" (Zhou et al., 2024). Additionally, each sample can be guided through different guidances, we applied this by uniformly distributing the guidance set $[0.01, 0.05, 0.1, 0.2, 0.3]$ to the samples.

A.3.2. DIFFUSION FORCING

We build our tested Diffusion Forcing upon the officially released code from <https://github.com/buoyancy99/diffusion-forcing>. The following modifications were made:

- **Replanning Frequency:** Since Diffusion Forcing inherently supports test-time replanning, we evaluated it under comparable conditions by applying periodic plan refinement, similar to Diffuser-Replanning.
- **Transformer Backbone:** To maintain consistency across models, we utilized the Transformer-based backbone provided in the Diffusion Forcing repository, rather than alternative architectures.

A.3.3. MONTE CARLO TREE DIFFUSION (MCTD)

MCTD follows Diffusion Forcing hyperparameters, but some of them are modified for MCTD and tree search related hyperparameters are added.

A.3.4. VALUE-LEARNING POLICY

For some tasks, to handle complex action space, we applied a value-learning policy by modifying the PlanDQ (Chen et al., 2024b) source code, <https://github.com/changchencc/plandq>.

A.4. The Evaluation Details

We followed the OGBench (Park et al., 2024) task configurations, 5 tasks per each environment. For each task, we evaluated 10 random seeds per each model, and reported the averaged success rates and their standard deviations.

A.5. Maze Navigation with Point-Mass and Ant Robots

A.5.1. MODIFICATIONS FROM OGBENCH

Although the original OGBench datasets include random noise in start and goal positions, we remove this noise to isolate model performance from environmental randomness. This modification facilitates clearer comparisons across methods and ensure that differences in performance can be attributed to planning capabilities rather than uncontrolled stochasticity.

Table 5. Diffusion Forcing Hyperparameters

Hyperparameter	Value
Learning Rate	$5e - 4$
Weight Decay	$1e - 4$
Warmup Steps	10000
Precision in Training	16-mixed
Precision in Inference	32
Batch Size	1024
Max Training Steps	200005
The Number of Frame Stack	10
Planning Horizon	Different for each task, please check in the task sections
Open Loop Horizon	50
Causal Mask	Not Used
Guidance Scale	3 for medium mazes and 2 for other mazes
Scheduling Matrix	pyramid
Stabilization Level	10
Beta Schedule	Linear
Diffusion Model Objective	x_0 -prediction
DDIM Sampling eta	0.0
Network Size	128
The Number of Layers	12
The Number of Attention Heads	4
The Feedforward Network Dimension	512

Additionally, we incorporate velocity into the state representation, which the original benchmark does not provide, in order to apply the heuristic controller for pointmaze tasks implemented in (Janner et al., 2022). For antmaze giant task, we found there are some cases where the plans are well generated, but due to the inaccurate execution, it cannot be succeed. To prevent this, we extend the episode length from 1000 to 1500.

A.5.2. GUIDANCE FUNCTION

To plan long-horizon while sustaining the plan effectiveness to reach the goal as fast as possible, we applied the guidance function to reduce the distances between every state and goal, $\sum_i ||x_i - g||_2$ (Chen et al., 2024a). We note that, to do fair comparison we applied the same guidance style over every baseline. For example, Diffuser originally used the goal-inpainting guidance for pointmaze, but in this evaluation, we used the distance guidance.

A.5.3. GUIDANCE SET

We applied different guidance sets to MCTD for the different tasks. For pointmaze medium and large, $[0, 0.1, 0.5, 1, 2]$ is applied, and for pointmaze giant, $[0.5, 1, 2, 3, 4]$ is applied. For antmaze tasks, $[0, 1, 2, 3, 4, 5]$ is set.

A.5.4. PLANNING HORIZON

For medium and large mazes, we applied 500 planning horizons and 1000 for giant to train the models with enough data. Medium and large *navigate* dataset trajectory horizons are 1000 and 2000 for giant. To make more data through sliding window technique, we applied smaller planning horizon than the dataset horizon.

A.5.5. HEURISTIC CONTROLLER AND VALUE-LEARNING POLICY

For pointmaze, we take the heuristic controller designed in (Janner et al., 2022). For antmaze, we use the value-learning policy (Wang et al., 2022) as done in (Chen et al., 2024b). However, we note that different from (Chen et al., 2024b), we only give the 10 steps further planned position information as the subgoal for that.

Table 6. MCTD Hyperparameters

Hyperparameter	Value
Learning Rate	$5e - 4$
Weight Decay	$1e - 4$
Warmup Steps	10000
Precision in Training	16-mixed
Precision in Inference	32
Batch Size	1024
Max Training Steps	200005
The Number of Frame Stack	10
Planning Horizon	Different for each task, please check in the task sections
Open Loop Horizon	Same to planning horizon for MCTD, 50 for MCTD-Replanning
Causal Mask	Not Used
Scheduling Matrix	pyramid
The Maximum Number of Search	500
Guidance Set	Different for each task, please check in task sections
The number of Partial Denoising	20
The Jumpy Denoising Interval	10
Stabilization Level	10
Beta Schedule	Linear
Diffusion Model Objective	x_0 -prediction
DDIM Sampling eta	0.0
Network Size	128
The Number of Layers	12
The Number of Attention Heads	4
The Feedforward Network Dimension	512

A.5.6. MCTD REWARD FUNCTION

For the reward function, we designed heuristic rules. One of those is to check whether there exists too large position difference between near states, because it is impossible physically. Another rule is to give a reward when reaching the goal. Earlier reaching can get the larger reward, $r = (H - t)/H$, where H and t are the horizon length and current step.

A.5.7. SHORT-HORIZON (STITCH) RESULTS

To examine performance when only short-horizon trajectories are given in the training phase, we evaluate on the stitch datasets, summarized in Table 8. The trajectory horizon in stitch datasets is much shorter than the required steps to reach the goal (e.g., the horizon in the dataset is 100 while over 200 steps are required in medium map tasks), we applied replanning on Diffuser, Diffusion Forcing, and MCTD. Different from the long-horizon dataset given cases, the Diffuser shows better performances than Diffusion Forcing, it can be seen that the implicit stitching ability of the Diffuser is better than Diffusion Forcing. On the other hand, MCTD shows better performance than baselines in the medium-sized map, due to multiple samplings with exploration, but the search length could not be long due to the limited length of trajectories in the training dataset, so it does not show good performance in large map.

A.5.8. COMPARISON WITH GOAL-INPAINTING DIFFUSER

Diffuser (Janner et al., 2022) typically employs a learned guidance function to bias generated trajectories toward high-return outcomes. For pointmaze tasks, however, an alternative approach—*goal-inpainting guidance*—has also been introduced (Janner et al., 2022), which further improves single-pass planning by “imagining” an intermediate trajectory whose final state is the designated goal. In the context of Diffuser-Replanning, we adapt goal-inpainting by providing the experienced trajectory as contextual information rather than a direct target for denoising.

As shown in Table 9, goal-inpainting outperforms standard Diffuser with heuristic guidance across medium, large, and giant

Table 7. Value-Learning Policy Hyperparameters

Hyperparameter	Value
Learning Rate	$3e - 4$
Learning Eta	1.0
Max Q Backup	False
Reward Tune	cql_antmaze
The Number of Training Epochs	2000
Gradient Clipping	7.0
Top-k	1
Target Steps	10
Randomness on Data Sampling (p)	0.2

Table 8. **Short-Horizon (Stitch) Maze Results.** Success rates (%) on reduced-horizon *stitch* variants of the pointmaze environment.

Dataset	Diffuser-Replanning	Diffusion Forcing	MCTD-Replanning
pointmaze-medium-stitch-v0	76 ± 12	53 ± 16	90 ± 14
pointmaze-large-stitch-v0	34 ± 16	20 ± 0	20 ± 0

pointmazes. By treating the goal as the terminal state to inpaint, the Diffuser model effectively samples trajectories from both ends, leveraging local convolutional mechanisms to reconcile the start and goal conditions. Although this approach can yield marked performance gains, **MCTD** remains superior in long-horizon scenarios, outperforming both the base and replanning variants of goal-inpainting Diffuser. The key distinction lies in MCTD’s tree-structured partial denoising and adaptive branching: while goal inpainting enhances single-pass generation, it lacks the iterative exploration-exploitation loop of explicit tree search. Moreover, the advantage of two-ended trajectory imagination diminishes when the maze is extremely large (e.g., giant mazes). Consequently, MCTD uncovers robust solutions even where goal inpainting begins to show diminishing returns, underscoring the benefits of tree-based refinement in complex, long-range planning tasks.

A.6. Robot Arm Cube Manipulation

A.6.1. OBJECT-WISE GUIDANCE

Because the robot arm can only move a single cube at any given time, we adopt an *object-wise guidance* mechanism in MCTD. Rather than applying a single holistic diffusion schedule to all cubes simultaneously, MCTD treats the selection of which cube to move next as a meta-action. This design partially alleviates issues that arise when a single plan attempts simultaneous motion of multiple cubes, though multi-object manipulation still demands careful sequencing to avoid suboptimal interleavings.

A.6.2. GUIDANCE FUNCTION

The guidance function itself is same to that for maze tasks, Appendix A.5.2, but the object-specific guidance is applied to MCTD and MCTD-Replanning. The guidance tries to reduce the distances between *specific object states and the object’s goal only*.

A.6.3. GUIDANCE SET

For cube tasks, the guidance set $[1, 2, 4]$ for each object is applied. For example, for two cube tasks, the size of the guidance set is 6 with three guidance set for each object.

A.6.4. REWARD FUNCTION FOR ROBOT ARM CUBE MANIPULATION

In addition to the reward function for the maze environment, we added some rules not to generate unrealistic plans. One of them is not to move multiple objects simultaneously. The small noise on planning is okay, but if multiple objects move adequately, the reward function decides the plan is unrealistic and gives a 0 reward. Other rules are, that the final state of the

Table 9. **Comparison Results with Goal-Inpainting Diffuser.** Success rates (%) on pointmaze and antmaze environments with medium, large, and giant mazes for the *navigate* datasets.

Environment	Dataset	Diffuser		Goal-Inpainting Diffuser		MCTD
		Base	Replanning	Base	Replanning	
pointmaze	medium-navigate-v0	58 $\pm$ 6	60 $\pm$ 0	84 $\pm$ 8	80 $\pm$ 9	100 $\pm$ 0
	large-navigate-v0	44 $\pm$ 8	40 $\pm$ 0	94 $\pm$ 9	84 $\pm$ 17	98 $\pm$ 6
	giant-navigate-v0	0 $\pm$ 0	0 $\pm$ 0	30 $\pm$ 16	34 $\pm$ 21	100 $\pm$ 0
antmaze	medium-navigate-v0	36 $\pm$ 15	40 $\pm$ 18	100 $\pm$ 0	94 $\pm$ 9	100 $\pm$ 0
	large-navigate-v0	14 $\pm$ 16	26 $\pm$ 13	86 $\pm$ 9	66 $\pm$ 13	98 $\pm$ 6
	giant-navigate-v0	0 $\pm$ 0	0 $\pm$ 0	12 $\pm$ 10	20 $\pm$ 0	94 $\pm$ 9

plan cannot be in the air, it should be on top of another cube or on the floor. The cube cannot posit where another cube is located, and when there is another cube on top of the target cube, the plan to move that is determined as unrealistic.

A.6.5. PLANNING HORIZON

For single cube tasks, the planning horizon is 200 same to the episode length, and 500 for others to train the models with enough data.

A.6.6. VALUE-LEARNING POLICY

Similar to antmaze tasks, we applied value-learning policy (Wang et al., 2022) by achieving the subgoals suggested by planners for 10 steps further planned states.

A.7. Visual Pointmaze

A.7.1. SETUP

Because diffusion-based planners typically operate on state-action representations, we first learn an *image encoder* via a Variational Autoencoder (VAE) (Kingma & Welling, 2013). The resulting latent states serve as inputs to the diffusion models. Notably, these latent encodings do not include explicit positional information, so the planner must infer underlying geometry and navigate the maze indirectly. We design an *inverse dynamics model* (a small network of linear layers) to convert consecutive latent observations into actions: $\hat{a}_t = \text{InvDyna}((x_{t-1}, x_t), x_{t+1})$. This replaces the heuristic controller of the original Diffuser, which cannot be directly applied here due to partial observability.

A.7.2. GUIDANCE

To guide the diffusion process, we introduce a *position estimator* trained on ground-truth coordinates. Although this estimator provides an approximate distance to the goal, the image encoder is not updated from this learning signal and it does *not* supply the planner with the full underlying state. Therefore, the image encoder still sees only the pixel-level data without direct positional labels, thereby preserving partial observability in the core planning procedure. This design choice highlights an open-ended question of how best to incorporate task-relevant signals in visual domains. The same guidance function with maze task, Appendix A.5.2 is applied.

A.7.3. GUIDANCE SET

For this task, we applied $[0, 0.1, 0.5, 1, 2]$ guidance set for MCTD and MCTD-Replanning.

A.7.4. DATASET GENERATION AND PLANNING HORIZON

We follow the OGBench (Park et al., 2024) dataset generation script for PointMaze, with trajectory horizons of 1000 for Medium and Large mazes. To enhance training, we use reduced planning horizons (500 for Medium/Large) and apply a sliding-window technique, as in our earlier pointmaze setup.

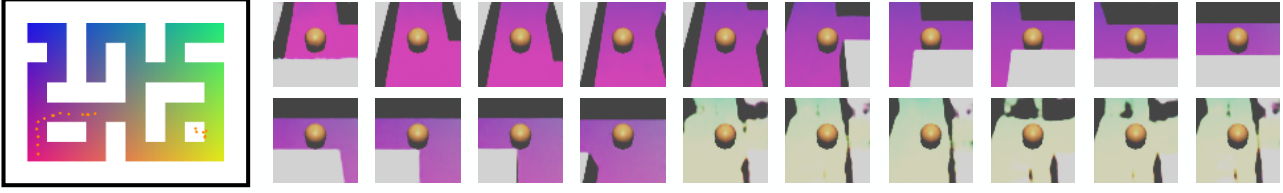


Figure 7. Trajectory collapse in Diffusion Forcing without replanning in the visual pointmaze. The left panel shows the estimated position of the planned trajectory (yellow points) halting at a certain area. The right panels (top left to bottom right) visualize the planning progression over time, suddenly teleporting to an unknown area. This underscores the importance of replanning in overcoming single-pass planning limitations.

A.7.5. VARIATIONAL AUTOENCODER

We pretrained a VAE (Kingma & Welling, 2013) to encode $64 \times 64 \times 3$ RGB observations into an 8-dimensional latent space, compressing high-dimensional inputs into a structured representation for downstream planning. It represents the latent variable z as a Gaussian distribution with mean μ and standard deviation σ , using the reparameterization trick:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (5)$$

We utilized this sampled latent embedding z as the input representation for downstreaming tasks.

A.7.6. INVERSE DYNAMICS MODEL AND POSITION ESTIMATOR

We trained Inverse Dynamics and Position Estimator models using an offline dataset encoded by the pretrained VAE. Both models are implemented as Multi-Layer Perceptrons (MLPs).

Inverse Dynamics. This model predicts the action $\hat{a}_t$ required to transition from the current state x_t to the next state x_{t+1} . To incorporate velocity-related information in a *partially observable* environment, the model takes both the previous state x_{t-1} and the current state x_t as inputs. The action is predicted as follows:

$$\hat{a}_t = \text{InvDyna}((x_{t-1}, x_t), x_{t+1}) \quad (6)$$

For the initial state, where x_{t-1} is unavailable, we use x_t twice as a substitute.

Position Estimator. The Position Estimator predicts the agent’s current position in the latent space encoded by the VAE, providing a weak positional signal to aid planning while preserving partial observability.

A.7.7. REPLANNING IS NECESSARY TO SOLVE COMPLEX ENVIRONMENTS

In this subsection, we investigate the failure modes of diffusion-based planners, particularly Diffusion Forcing without replanning, in solving the visual pointmaze task. Figure 7 demonstrates how the absence of iterative corrections leads to trajectory collapse, revealing critical challenges in pixel-level planning. These findings indirectly highlight the necessity of replanning for robust trajectory generation in complex and partially observable environments.

B. MCTD Architecture Implementation

The Monte Carlo Tree Diffusion (MCTD) framework relies on three key elements within its tree-search loop: sub-plan noise scheduling, causal semi-autoregressive denoising, and partial, jumpy denoising. First, rather than assigning a single noise schedule to the entire trajectory, each sub-trajectory segment receives its own noise-level schedule. This design allows different segments to transition from high to low noise at distinct rates, thereby enabling focused refinement of earlier parts of the plan while preserving flexibility in later segments.

Next, we adopt a *causal noise schedule* (Chen et al., 2024a), which renders the denoising process semi-autoregressive: earlier plan segments undergo more thorough denoising, while subsequent ones shift more gradually from higher to lower

noise. This preserves the globally consistent trajectory generation characteristic of diffusion models but also ensures that critical decisions near the beginning of the plan receive higher-fidelity refinements. Finally, both planning and simulation employ partial and jumpy denoising via DDIM sampling (Song et al., 2020a), allowing the system to skip directly from noise level t to $t - \Delta$. This skip-level update accelerates the evaluation of candidate trajectories during tree expansion and supports partial denoising steps essential for exploration.

We implement these strategies with a Transformer-based model that processes an *expandable* token sequence, thus avoiding any architectural constraint on the planning horizon. Each sub-plan is annotated with a distinct noise-level index, and the Transformer predicts the fully denoised data x_0 from the corresponding noisy tokens. Concretely, we employ an x_0 -parameterization by training the network to minimize the squared error between the predicted $\hat{x}_0$ and the ground-truth x_0 :

$$\mathcal{L}_{\text{train}} = \mathbb{E}_{x_0, t, \epsilon} [\|\hat{x}_0 - x_0\|^2]. \quad (7)$$

Because each sub-plan has its own noise schedule, the Transformer naturally accommodates varying horizon lengths and partial denoising intervals without requiring further modifications to the model architecture.

By integrating causal scheduling with skip-level DDIM updates, MCTD achieves a semi-autoregressive, partial denoising process within a tree-search loop. Early plan segments are refined immediately, while future segments remain at higher noise until exploration or heuristic signals indicate their importance. This arrangement allows for fast approximate rollouts by skipping multiple noise levels when simulating a plan. Consequently, MCTD integrates the holistic generative capacity of diffusion models with the adaptive, branch-and-bound nature of Monte Carlo Tree Search, thereby enabling iterative trajectory refinement in tasks with long horizons and sparse rewards.

C. Detailed Comparison between Monte Carlo Guidance (MCG) and Monte Carlo Tree Diffusion (MCTD)

While **MCG (Monte Carlo Guidance)** utilizes Monte Carlo sampling to adjust guidance levels during denoising, it lacks the structured search capabilities of **MCTD (Monte Carlo Tree Diffusion)** and does not explicitly refine trajectories over time. MCG focuses on adjusting guidance strengths rather than performing an explicit **trajectory search**, making it less effective for long-horizon planning where iterative improvement is crucial. Additionally, **MCG does not implement the four standard MCTS steps (Selection, Expansion, Simulation, and Backpropagation)**, limiting its ability to efficiently allocate test-time compute (TTC) for progressive refinement. Without a structured search process, **MCG cannot revisit and optimize subplans**, making it less scalable for complex decision-making tasks. Furthermore, MCG lacks **meta-actions for adaptive exploration-exploitation**, passively adjusting guidance instead of dynamically balancing search strategies. Finally, it does not incorporate **fast jumpy denoising** for simulation, making trajectory evaluation less efficient compared to MCTD, which efficiently estimates plan quality without costly forward model rollouts.

D. Algorithms

Algorithm 2 Monte Carlo Tree Diffusion (MCTD)

```

1: procedure MCTD(root, iterations)
2:   for  $i = 1$  to iterations do
3:      $node \leftarrow \text{SELECTPROMISINGNODE}(root)$ 
4:     if  $\text{ISEXPANDABLE}(node)$  then
5:        $node \leftarrow \text{EXPANDNODE}(node)$ 
6:     end if
7:      $reward \leftarrow \text{SIMULATE}(node)$ 
8:      $\text{BACKPROPAGATE}(node, reward)$ 
9:   end for
10:  return  $\text{BESTCHILD}(root)$ 
11: end procedure

```

Algorithm 3 Selection in MCTD

```

1: procedure SELECTPROMISINGNODE(node)
2:   while ISFULLYEXPANDED(node) and not ISLEAF(node) do
3:     node  $\leftarrow$  BESTUCTCHILD(node) {Use UCB for exploration-exploitation}
4:   end while
5:   return node
6: end procedure

```

Algorithm 4 Expansion in MCTD

```

1: procedure EXPANDNODE(node)
2:   gs  $\leftarrow$  SELECTMETACTION(node) {Determine guidance level}
3:   child  $\leftarrow$  DENOISESUBPLAN(node, gs) {Generate new subplan using diffusion}
4:   ADDCHILD(node, child)
5:   return child
6: end procedure

```

Algorithm 5 Simulation in MCTD (Jumpy Denoising)

```

1: procedure SIMULATE(node)
2:   fullPlan  $\leftarrow$  FASTJUMPYDENOISING(node)
3:   return EVALUATEPLAN(fullPlan)
4: end procedure

```

Algorithm 6 Backpropagation in MCTD

```

1: procedure BACKPROPAGATE(node, reward)
2:   while node  $\neq$  null do
3:     node.visitCount  $\leftarrow$  node.visitCount + 1
4:     node.value  $\leftarrow$  node.value + reward
5:     UPDATEMETACTIONSCHEDULE(node, reward) {Adjust guidance levels}
6:     node  $\leftarrow$  node.parent
7:   end while
8: end procedure

```

Algorithm 7 Meta-Action and Guidance Selection

```

1: procedure SELECTMETACTION(node)
2:   return UCBSELECTION({NO, LOW, MEDIUM, HIGH})
3: end procedure
4: procedure DENOISESUBPLAN(node, gs)
5:   if gs = NO then
6:     return SAMPLEPRIOR( $p(\mathbf{x}_s | \mathbf{x}_{1:s-1})$ ) {Exploration}
7:   else
8:     return SAMPLEGUIDED( $p_g(\mathbf{x}_s | \mathbf{x}_{1:s-1})$ ) {Exploitation}
9:   end if
10: end procedure

```

Algorithm 8 Jumpy Denoising for Fast Simulation

```

1: procedure FASTJUMPYDENOISING(partialPlan)
2:   return SAMPLE( $p(\mathbf{x}_{s+1:S} | \mathbf{x}_{1:s})$ ) {X-shot fast decoding}
3: end procedure

```

Algorithm 9 Partial and Jumpy Denoising with DDIM

Require: Initial plan $\mathbf{x}_{t_0}$ at noise level t_0 , noise schedule $\mathcal{N}' = [t_0, t_1, \dots, t_K]$, diffusion model $f_\theta(\cdot)$, guidance scale g

Ensure: Partially denoised plan $\mathbf{x}_{t_K}$

```

1: for  $k = 0, 1, \dots, K - 1$  do
    // Predict noise or score at level  $t_k$ 
2:    $\epsilon \leftarrow f_\theta(\mathbf{x}_{t_k}, t_k)$  {DDIM  $\epsilon$ -prediction or score function}
    // Apply guide using gradients of return
3:    $\epsilon^{(\text{guided})} \sim \mathcal{N}(\epsilon + \alpha \Sigma \nabla \mathcal{J}, \Sigma^t)$ 
    // DDIM update from  $t_k$  to  $t_{k+1}$ 
4:    $\mathbf{x}_{t_{k+1}} \leftarrow \sqrt{\frac{\alpha_{t_{k+1}}}{\alpha_{t_k}}} \left( \mathbf{x}_{t_k} - \sqrt{1 - \alpha_{t_k}} \epsilon^{(\text{guided})} \right) + \sqrt{1 - \alpha_{t_{k+1}}} \epsilon^{(\text{guided})}$ 
5: end for
6: return  $\mathbf{x}_{t_K}$ 

```

Algorithm 10 Additional Controller/Policy/Inverse Dynamics Model Integration

Require: Long-term planner, P_θ , environment E and controller π_γ (heuristic controller or value learning policy or inverse dynamics model), planning horizon H

```

1:  $e \sim E$  with initial observation and goal  $s, g$ 
2: while not done do
    // Get a plan from planner  $P_\theta$ 
3:    $p_{1:H} = P_\theta(s, g)$ 
4:    $i = k$  {k steps after plan state is the subgoal for Controller}
5:    $g' = p_k$ 
6:   while  $i \leq H$  do
    // Compute low-level action with Controller
7:      $a \leftarrow \pi_\gamma(s, g')$ 
    // Interact with environment
8:      $(o, r, \text{done}) \leftarrow e.\text{step}(a)$ 
9:      $s \leftarrow o$ 
10:    if  $s \approx g'$  then
11:       $i = i + k$ 
12:       $g' = p_k$ 
13:    end if
14:    if done then
15:      break
16:    end if
17:  end while
18: end while

```
